Incident Management API (Spring Boot & Security)

Build a production-grade, multi-tenant incident tracking microservice implementing stateless JWT authentication, role-based database access, and optimistic locking.

1. DOMAIN & DATA ARCHITECTURE

Implement the following JPA entities with appropriate validation annotations and auditing fields (`createdAt`, `updatedAt`):

- **User:** `id`, `email` (unique), `password` (BCrypt encoded), `role` (`ROLE_USER`, `ROLE_ADMIN`).
- **Incident:** `id`, `title` (max 120 chars), `description`, `severity` (`LOW`, `MEDIUM`, `HIGH`, `CRITICAL`), `status` (`OPEN`, `IN_PROGRESS`, `RESOLVED`), `reportedBy` (User relation), `version` (JPA `@Version` for concurrency control).

2. ENDPOINT CONTRACT & RBAC

Method & Route	Access Level	Business Rules & Validation
`POST /api/v1/auth/token`	PUBLIC	Authenticates credentials against DB; returns JWT access token containing subject and authority claims.
`GET /api/v1/incidents`	USER ADMIN	Returns paginated incidents. `ROLE_USER` retrieves only self-reported incidents; `ROLE_ADMIN` fetches system-wide records. Supports sorting by severity.
`POST /api/v1/incidents`	USER	Creates a new incident. Automatically binds current security context user as `reportedBy`. Default status: `OPEN`.
`PATCH /api/v1/incidents/{id}`	USER ADMIN	Updates status/severity. Non-admins can only update incidents they created. Must handle concurrency via optimistic locking (`409 Conflict` on stale version).
`DELETE /api/v1/incidents/{id}`	ADMIN	Hard deletes an incident record. Restricts access strictly to administrators.

3. PRODUCTION ACCEPTANCE CRITERIA

- **Security Config:** Use Spring Security 6+ Lambda DSL. Disable CSRF, set session management to `STATELESS`, and register custom JWT filter prior to `UsernamePasswordAuthenticationFilter`.
- **Data Protection:** Write custom Spring Data JPA repository queries using JPQL or Specification API to enforce tenant isolation at database level.
- **Error Handling:** Implement `@RestControllerAdvice` to transform exceptions (`MethodArgumentNotValidException`, `EntityNotFoundException`, `AccessDeniedException`) into structured RFC 7807 Problem Details JSON format.